

Progetto Reinforcement Learning

Giacomo Baresi - 735878, Francesco Lozio - 737664

June 2026

Contents

1	Introduzione	3
2	Algoritmi	3
2.1	Q-Learning	3
2.2	SARSA	4
2.3	Monte Carlo Prediction	4
2.4	Temporal Difference	5
2.5	Deep Q-Network	5
2.5.1	Experience Replay	5
2.5.2	Target Network	6
3	Setup d'Environment	6
3.1	Cart Pole	6
3.2	Mountain Car	7
3.3	Freeway	7
4	Risultati	8
4.1	Cart Pole	8
4.1.1	Apprendimento Policy	8
4.1.2	Prediction	9
4.1.3	Sensibilità α	10
4.1.4	Sensibilità γ	11
4.1.5	Sensibilità ϵ	12
4.2	Mountain Car	14
4.2.1	Apprendimento Policy	14
4.2.2	Prediction	14
4.2.3	Sensibilità α	15
4.2.4	Sensibilità γ	16
4.2.5	Sensibilità ϵ	18
4.3	Freeway	19
5	Failures	21

6	Conclusioni	22
6.1	Aspettative	22
6.2	Le sorprese	22

1 Introduzione

In questa relazione vengono discussi gli algoritmi usati ed i risultati per l'apprendimento di policy in merito a tre ambienti utilizzati mediante libreria Gymnasium ed ALE-py:

- Cart Pole - in questo ambiente, l'obiettivo è quello di non far cadere un bastone posto in equilibrio su una pedana mobile;
- Mountain Car - in questo ambiente, l'obiettivo è quello di far muovere la macchina in una depressione per arrivare alla fine di una salita segnata da una bandiera;
- Freeway - in questo ambiente, l'obiettivo è consentire al controllabile di raggiungere la parte opposta della strada, evitando le macchine che invece lo forzano ad indietreggiare rispetto al traguardo.

Le effettive implementazioni delle librerie vengono discusse nella sezione Setup d'Environment. Gli algoritmi usati sono i seguenti secondo quanto richiesto dalla consegna:

- Q-Learning
- SARSA
- Monte Carlo
- Temporal Difference (con $\lambda = 0$)
- Deep Q-Network

Questi algoritmi vengono approfonditi nella sezione Algoritmi.

2 Algoritmi

2.1 Q-Learning

Q-Learning è un algoritmo off-policy: l'agente continua ad interagire con l'environment ma l'aggiornamento della Q-table avviene dopo aver scelto l'azione. Ad ogni timestep, l'algoritmo sceglie la prossima azione (ad esempio, con epsilon-greedy) e dopo essere arrivato nel nuovo stato, aggiorna il valore Q-table ma dello stato e dell'azione scelti per arrivare nel nuovo stato. In particolare, la formula prevede di sommare al valore corrente α volte (learning rate) la somma tra il reward ottenuto con l'azione più γ volte (discount factor) il prossimo valore di Q table più alto scegliendo la migliore azione a' dallo stato s' , e questo sottratto del valore corrente di Q nello stato iniziale con l'azione scelta.

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \alpha_t \left(R_{t+1} + \gamma \max_{a'} q_t(S_{t+1}, a') - q_t(S_t, A_t) \right)$$

Figure 1: Formula Q-Learning: [6] Model-Free Control, slide 20

Questo consente di esplorare e solo successivamente valutare quanto lo stato in cui si trovava l'agente sia buono con quell'azione scelta. Questo consente una maggiore esplorazione degli stati. Da notare che l'azione migliore considerata per l'aggiornamento di Q non è necessariamente l'azione che l'agente sceglierà al prossimo timestep.

2.2 SARSA

SARSA è un algoritmo on-policy che aggiorna il valore $q(s,a)$ dopo che ha scelto l'azione anche nello stato successivo: partendo da uno stato S ed eseguendo un'azione A, otteniamo una reward R e finiamo nello stato S'. In questo stato, eseguiamo l'azione A'. Con questo, aggiorniamo la Q_{table} dello stato S con azione A sommando alpha volte la somma tra reward e gamma volte la Q di S' e A' meno il valore Q di S e A. Quindi, in sostanza, aggiorniamo Q in favore della reward effettivamente ottenuta e anche in base a quanto fare l'azione A mi ha aiutato finendo in S' e scegliendo A'. A differenza di Q-Learning quindi l'aggiornamento avviene seguendo la policy che si sta usando perché non viene più richiesto di usare l'azione che massimizza Q in S' ma si usa l'azione effettivamente scelta che potrebbe essere stata scelta casualmente anziché valutata con l'opzione migliore.

$$q_{t+1}(s,a) = q_t(S_t, A_t) + \alpha_t (R_{t+1} + \gamma q_t(S_{t+1}, A_{t+1}) - q_t(S_t, A_t))$$

Figure 2: Formula Q-Learning: [6] Model-Free Control, slide 18

2.3 Monte Carlo Prediction

Monte Carlo Prediction è un algoritmo di valutazione di policy che aggiorna una $V(s)$ che assegna ad ogni stato s un valore. Questa valutazione si basa sulla policy usata: Monte Carlo esplora gli stati con una policy π fino alla fine dell'episodio. Sulla base delle ricompense ottenute (sommate a ritroso) si ottiene il valore di ogni stato. Questo lo rende suscettibile in termini di varianza: una scelta casuale può portare a valori di reward non veritieri rispetto alla policy, portando a valutare gli stati non correttamente. Non soffre di bias però: essendo il valore calcolato usando strettamente i reward ottenuti, non ci sono bias dovuti ad errori dell'algoritmo stesso, rendendolo (teoricamente) un candidato migliore per policy senza scelte casuali.

2.4 Temporal Difference

Similmente a Monte Carlo Prediction, Temporal Difference valuta una policy e assegna il valore $v(s)$ ma la valutazione avviene dopo N step: se $N = 1$, la valutazione avviene immediatamente dopo aver eseguito un'azione. Nel nostro caso, usiamo $TD(0)$, ossia non abbiamo Eligibility Traces e procediamo con uno step alla volta. La formula per l'aggiornamento di $V(s)$ prevede l'aggiunta di alfa volte la reward ottenuta, sommata a gamma volte il $V(s')$ con s' nuovo stato e meno il $v(s)$ corrente. Temporal Difference (senza Eligibility Traces come usato nel progetto) soffre di bias: l'aggiornamento del $V(s)$ dipende dal valore dello stato successivo ($V(S_{t+1})$) che inizialmente è inizializzato a zero e che dipende anche dall'algoritmo, portando ad errori dovuti dall'algoritmo stesso. Soffre meno di problemi di varianza: dipendendo da un solo stato e da una sola reward, ci sono meno possibilità di ottenere risultati sfasati.

$$v_{t+1}(S_t) = v_t(S_t) + \alpha_t (R_{t+1} + \gamma v_t(S_{t+1}) - v_t(S_t))$$

Figure 3: Formula Q-Learning: [6] Model-Free Control, slide 18

2.5 Deep Q-Network

Per Freeway viene usata una Deep Q-Network. Quando l'Observation Space è rappresentato con immagini, è necessaria una CNN senza discretizzare: in questa implementazione, vengono usati tre layer di convoluzione intervallati da una batch normalization. La rete viene addestrata durante l'episodio. La peculiarità della Deep Q-Network è una loss differente, in particolare è:

$$L(\mathbf{w}) = \frac{1}{2} \left(R_{t+1} + \gamma \left[\max_a q_{\mathbf{w}}(S_{t+1}, a) \right] - q_{\mathbf{w}}(S_t, A_t) \right)^2$$

Figure 4: Formula Deep Q-Learning: [8] Deep RL, slide 14

Notare che questa formula di "loss" rispecchia quella usata in Q-Learning ma adattata per essere una loss (seppur non lo sia, di fatto). In questo modo, i pesi consentono di mappare una Q-table. In aggiunta, vengono implementati Experience Replay e Target Network.

2.5.1 Experience Replay

Quando si interagisce in ambienti per Reinforcement Learning, i dati solitamente non sono i.i.d (Indipendenti ed Identicamente Distribuiti) e questo può creare problemi per l'apprendimento di una CNN. A tale scopo, viene introdotta la tecnica di Experience Replay: ad ogni iterazione, si salva la tupla Stato, Azione,

Reward, Stato successivo. Dopo ogni passo, se ci sono abbastanza tuple, se ne prendono N casuali (con N grandezza della batch, in questo caso ha un valore di default 32) e si computa la loss di questi sample, se ne fa una media e si usa per aggiornare i pesi. Questo consente di "rompere" la dipendenza temporale tra gli stati che esiste in contesti come quello di Freeway trasformando Deep Q-Network in un algoritmo off-policy.

2.5.2 Target Network

Con l'aggiunta di Experience Replay, ora Deep Q-Network è Off-Policy mentre la CNN fungeva da Function Approximation e la formula della loss faceva bootstrap scegliendo l'azione che massimizzava la Q moltiplicata per gamma. Questa combinazione è nota come deadly triad e causa Soft-Divergence, portando ad una sovrastima dei valori a causa dell'aggiornamento dei pesi continuo. Per ovviare a questo problema, la tecnica di Target Network prevede l'utilizzo di una seconda CNN che viene aggiornata solamente dopo un certo numero di step (in questo progetto si è optato per 20 step) ed usata per il bootstrapping: in questo modo, la deadly triad viene mitigata mentre la CNN originale viene aggiornata ad ogni step ed usata per loss e function approximation.

3 Setup d'Environment

Per l'utilizzo degli environment, sono state utilizzate le librerie Gymnasium[1] e ALE-py[2], in particolare Gymnasium è stato usato per Cart Pole e Mountain Car mentre per Freeway è stata necessaria l'aggiunta di ALE-py. Tutti gli studi sono fatti eseguendo gli appositi script *study_X.py* ma è possibile eseguire gli environment singolarmente.

3.1 Cart Pole

Nell'Environment Cart Pole-v1 di Gymnasium[3] l'Observation Space considera:

- La posizione del cart (valori tra -4.8 e 4.8);
- La velocità del cart (senza limiti);
- L'angolo del bastone (valori tra -24 e 24 gradi);
- La velocità angolare del bastone (senza limiti).

Le azioni dell'agente consentono di muovere il cart o verso sinistra o verso destra e per ogni step riceve una reward di +1 (quindi più l'agente riesce a lasciare il bastone in equilibrio e più verrà ricompensato). L'episodio si considera concluso se:

- L'angolo del bastone raggiunge 12 gradi, sia in senso orario che antiorario (Episodio Terminato)

- La posizione del cart raggiunge 2.4, sia in avanti che indietro (Episodio Terminato)
- L'episodio raggiunge i 500 step (Episodio Troncato)

Quando una di queste tre condizioni viene raggiunta, l'episodio è concluso. Se l'episodio viene troncato significa che l'algoritmo è stato in grado di "vincere" perché è riuscito a mantenere il bastone ed il cart entro il "tempo limite", segno che l'algoritmo ha imparato una policy sufficientemente buona (oppure è stato molto fortunato). Essendo lo spazio degli stati troppo ampio a causa dei valori continui dell'Observation Space, si discretizza lo spazio dividendo ogni dimensione dell'Observation Space in intervalli. Nello specifico di questa implementazione, i valori sono divisi in 24 intervalli uniformi tranne l'angolo del bastone che è diviso in 30 per avere più precisione, avendo un requisito più stretto e meno controllabile.

3.2 Mountain Car

Nell'Environment Mountain Car di Gymnasium[4] l'Observation Space consta di 2 elementi:

- Posizione della macchina lungo l'asse X (valori tra -1.2 e 0.6);
- Velocità della macchina (valori tra -0.07 e 0.07).

L'agente può scegliere se accelerare verso destra, verso sinistra o non accelerare affatto. L'obiettivo dell'agente è quello di raggiungere la bandiera sulla collina a destra il prima possibile e quindi, ad ogni timestep, viene assegnata una reward di -1. L'episodio si considera concluso se:

- La macchina raggiunge la bandiera, che si traduce in posizione sull'asse X maggiore o uguale di 0.5 (Episodio Terminato);
- L'episodio raggiunge i 200 step (Episodio Troncato);

Se l'episodio viene terminato significa che l'agente ha "vinto" perché ha raggiunto il suo goal entro il tempo limite il che significa che la sua policy è sufficientemente buona per vincere. Anche in questo Environment, è necessario discretizzare l'Observation Space: nel nostro caso, si è optato per dividere entrambe le dimensioni in 24 intervalli.

3.3 Freeway

Per utilizzare Freeway è necessaria la libreria ALE-py[5]. A differenza dei precedenti Environment, l'Observation Space di Freeway è composto da immagini RGB 210x160 (seppur modificabile con modalità "ram" o "grayscale" per ridurne la dimensionalità). Quindi, al fine di elaborare questa osservazione, l'immagine non viene discretizzata ma passa per la CNN del Deep Q-Network. L'agente può scegliere se muoversi in su, giù o non muoversi. Ogni volta che

l'agente riesce a raggiungere il lato opposto della strada, riceve un punto (reward +1). L'episodio termina dopo 2 minuti e 30 secondi e l'agente deve fare quanti più punti possibili in questo tempo limite. Il tempo di 2 minuti e 30 secondi scorre diversamente quando non si usa la rappresentazione grafica: il processore non è limitato dal gioco stesso e può andare quanto più veloce possibile, consentendo di concludere un episodio in una manciata di secondi (con GPU)

4 Risultati

In questa sezione vengono illustrati i risultati ottenuti. Per gli Environment di Cart Pole e Mountain Car si analizzano i risultati di SARSA, Q-Learning, Monte Carlo e Temporal Difference mentre per Freeway si usa solo Deep Q-Network. I risultati vengono da medie su 3 seed diversi e le bande rappresentano la deviazione standard. Essendo Monte Carlo e Temporal Difference algoritmi basati sulla costruzione della Value Function $V(S)$, non c'è alcuna policy da imparare e quindi le learning curves sarebbero poco informative. Invece, vengono mostrate heatmap per mostrare i valori calcolati usando delle policy ad hoc per entrambi gli ambienti. I valori degli iperparametri, quando non variano, sono:

- $\alpha = 0.2$
- $\gamma = 0.99$
- $\epsilon = 0.1$

4.1 Cart Pole

4.1.1 Apprendimento Policy

Come si evince in 5, sia Q-Learning che SARSA imparano una policy che gli consente mano a mano di tenere in equilibrio il bastone sempre più a lungo ma non abbastanza da raggiungere il limite di 500 episodi. In generale, Q-Learning sembra dimostrarsi più efficace rispetto a SARSA ma, vista la bassa stabilità, non è un'osservazione ritenuta abbastanza supportata.

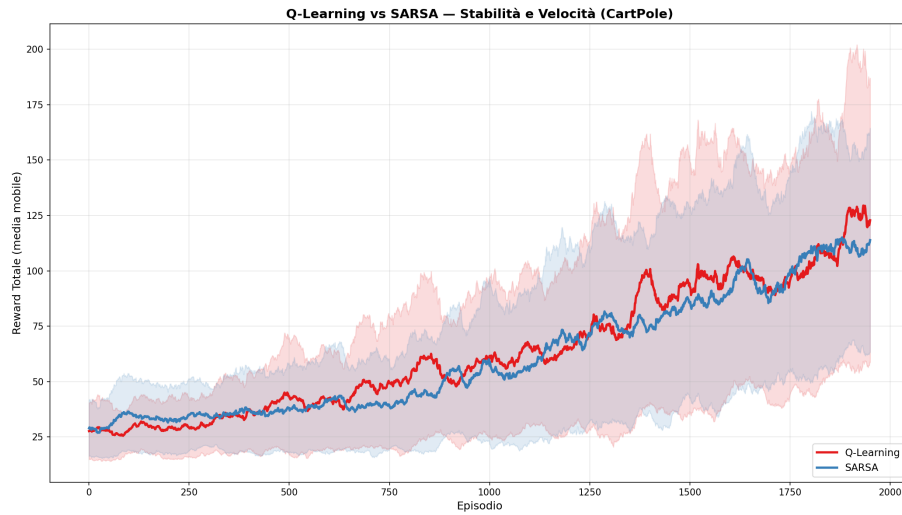


Figure 5: Confronto SARSA e Q-Learning

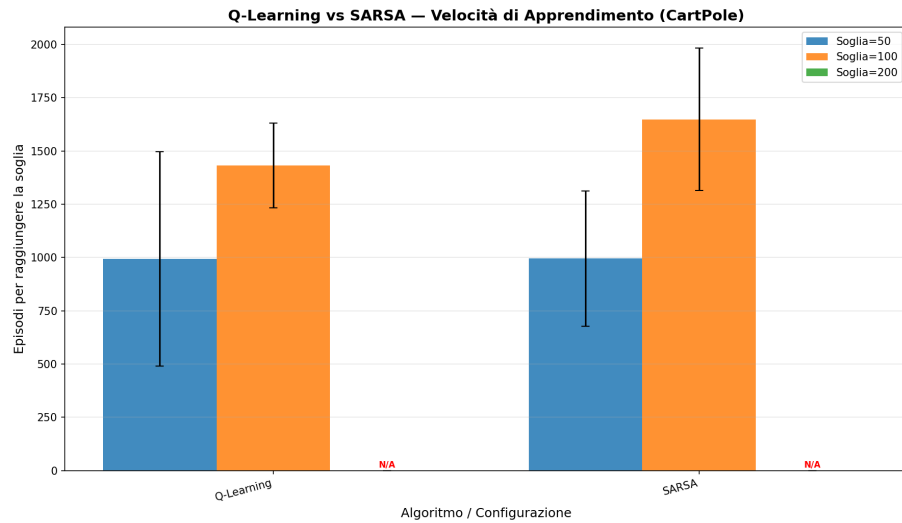


Figure 6: Confronto Velocità SARSA e E-Learning

4.1.2 Prediction

Per Cart Pole, la policy usata per Monte Carlo e Temporal Difference era molto semplice: quando il bastone sta cadendo verso destra (cfr. angolo positivo), spostalo verso sinistra. Altrimenti, l'azione è di spostare il bastone verso destra. Questa semplice policy viene valutata dagli algoritmi: entrambe le Value

Function considerano i medesimi stati, entrambi hanno correttamente visitato gli stessi stati mentre gli altri avevano valore nullo.

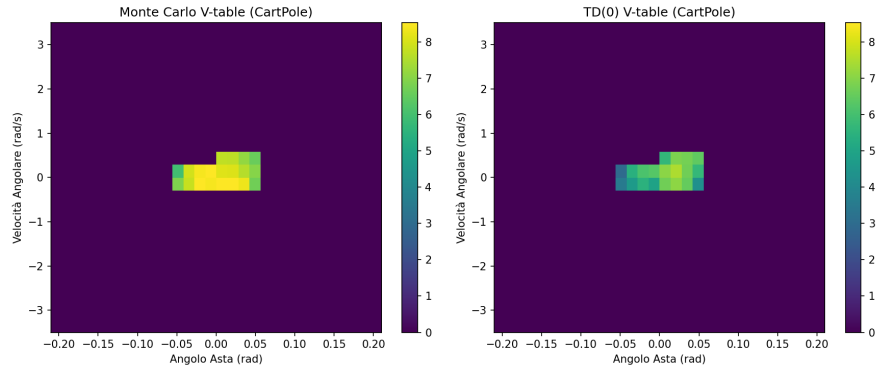


Figure 7: Heatmap MC vs TD

4.1.3 Sensibilità α

Con un α più alto, sia SARSA che Q-Learning sembrano apprendere meglio: incrementando α oltre 0.1 si nota infatti un aumento del reward totale, indicando che la policy appresa dall'agente si trova ancora lontana da quella ottima, favorendo grandi "salti" in avanti che portano prima a reward migliori.

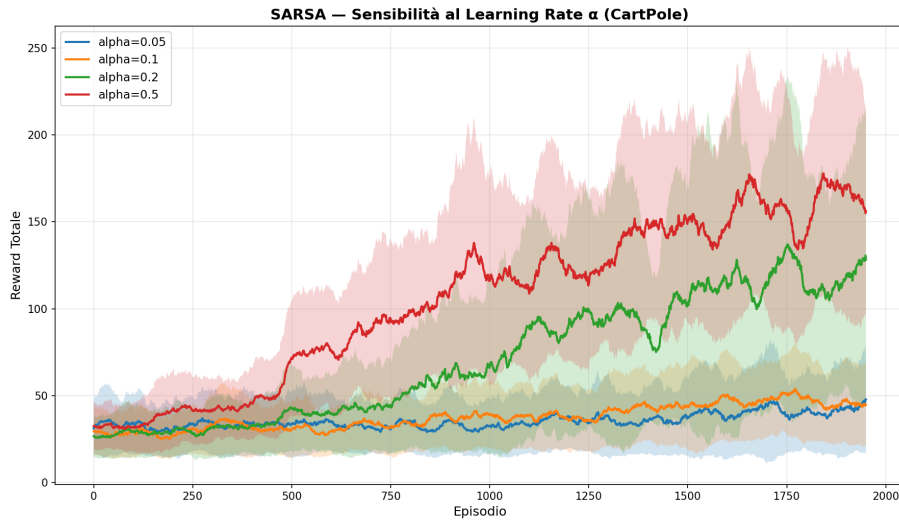


Figure 8: Sensibilità SARSA α

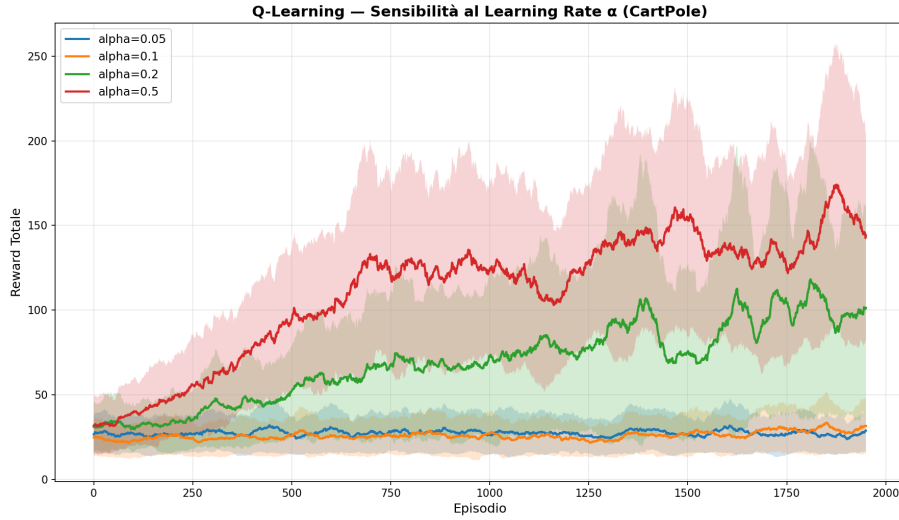


Figure 9: Sensibilità Q-Learning α

4.1.4 Sensibilità γ

In SARSA, la variazione di discount factor si dimostra significativa solo con valore pari a 1: far valere la prima azione come l'ultima sembra avere un impatto significativo sull'algoritmo che lo porta a raggiungere quasi 250 di reward totale. Anche Q-Learning migliora con un discount factor più alto ma non come SARSA: Q-Learning mostra migliori già con γ più bassi e non raggiunge il picco di SARSA. A livello logico, far valere di più le azioni passate ha senso in Cart Pole: l'azione che mi ha portato in uno stato potenzialmente a rischio terminazione è influenzata da quella prima. Se per esempio continuo a spingere il mio bastone a destra, lo porto pericolosamente vicino all'angolo limite e quando sono vicino alla terminazione non posso non tenere conto di questa pessima decisione.

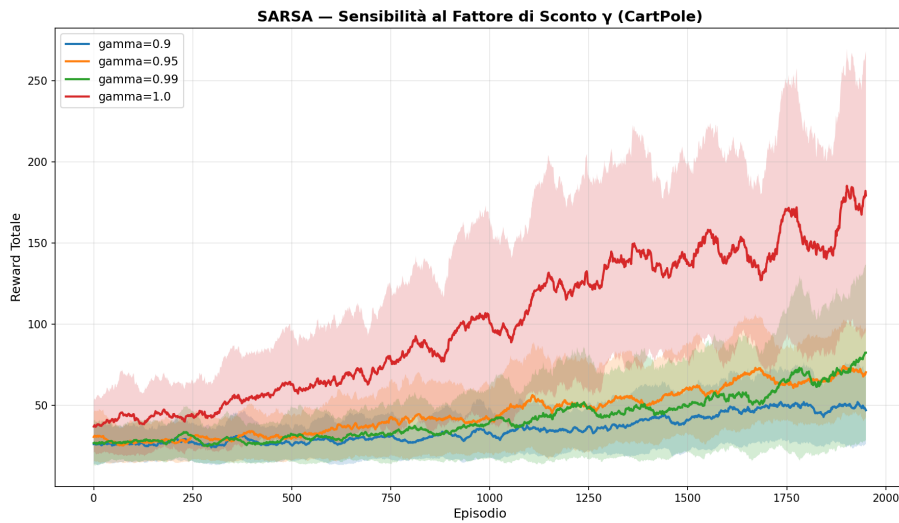


Figure 10: Sensibilità SARSA γ

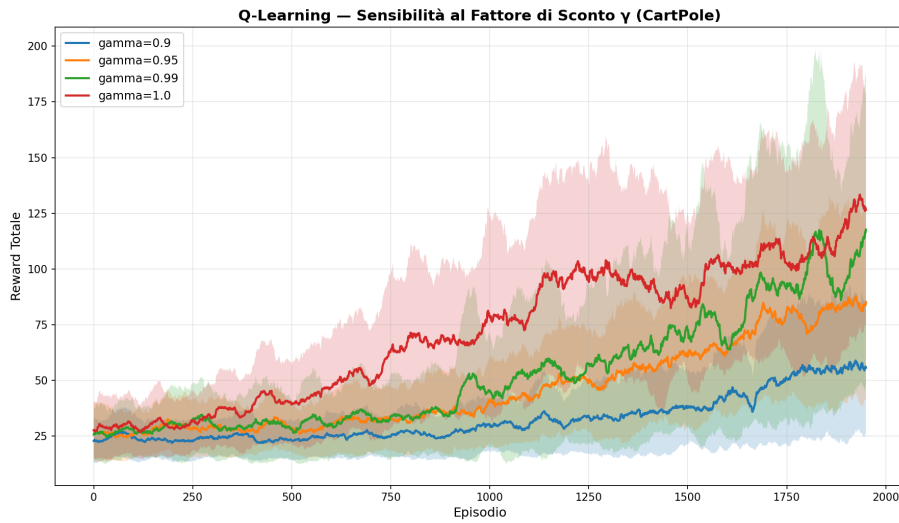


Figure 11: Sensibilità Q-Learning γ

4.1.5 Sensibilità ϵ

Entrambi gli algoritmi sembrano preferire un ϵ più grande: gli agenti ottengono un reward più alto quando fanno mosse casuali. Data la natura di Cart Pole però risulta difficile verificare quanto sia l'effettivo impatto: anzitutto, stiamo

parlando di probabilità, un fattore che da sé è difficile da studiare. In secondo luogo, in Cart Pole ci sono solamente 2 azioni, scegliendo casualmente abbiamo comunque il 50% di scegliere l'azione migliore.

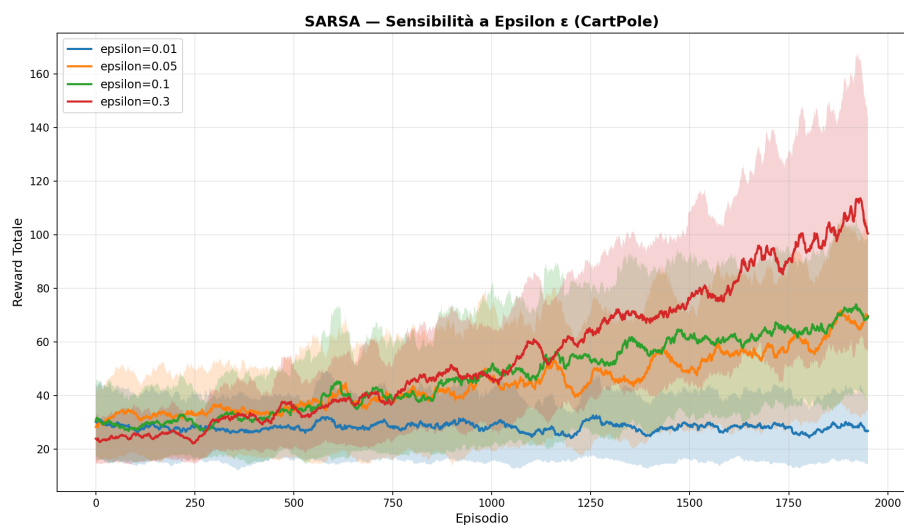


Figure 12: Sensibilità SARSA ϵ

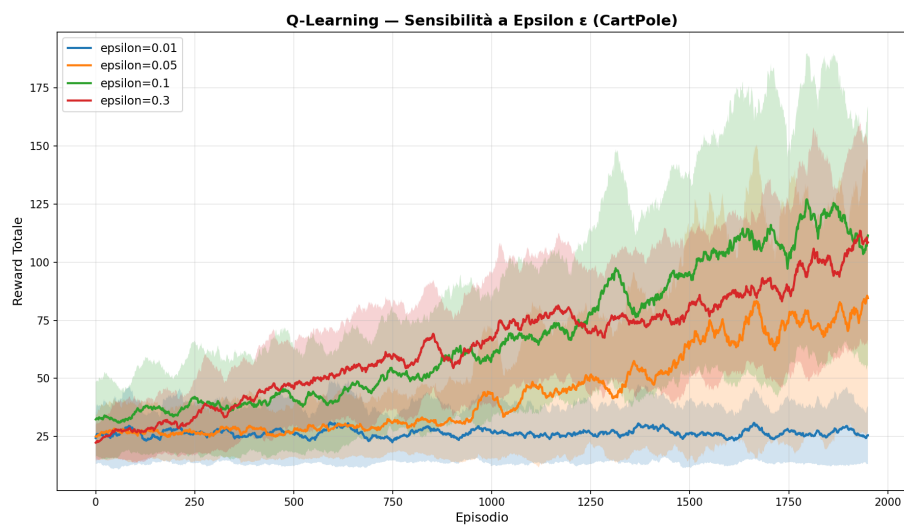


Figure 13: Sensibilità Q-Learning ϵ

4.2 Mountain Car

4.2.1 Apprendimento Policy

In Mountain Car, i risultati sembrano oscillare di più: sia Monte Carlo che SARSA raggiungono alcuni valori di picco senza un'apparente pattern e anche le loro deviazioni standard mostrano una certa instabilità. SARSA sembra apprendere "più velocemente" una policy appena sufficiente per riuscire a vincere entro i timestep limite.

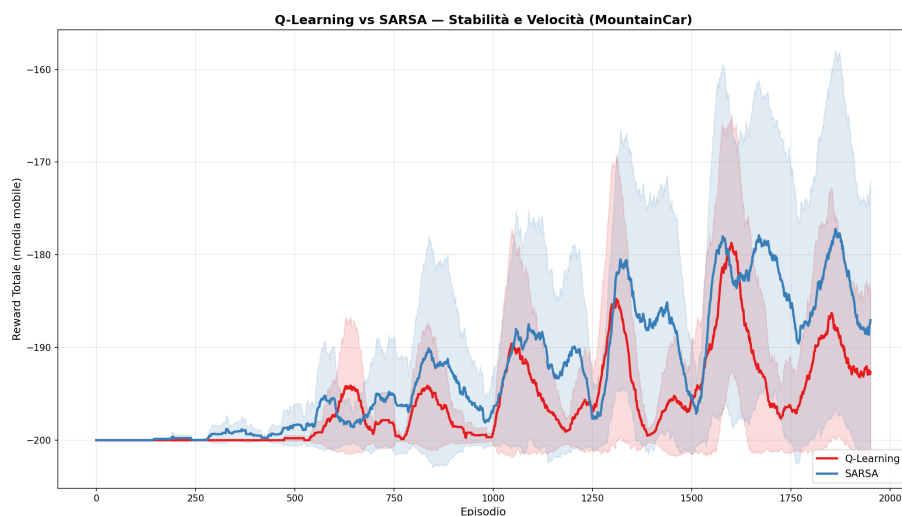


Figure 14: Confronto SARSA e Q-Learning

4.2.2 Prediction

La policy usata per la valutazione della Value Function in Mountain Car è la seguente:

- Se la velocità è negativa, spingi verso destra
- Se la velocità è positiva, spingi verso sinistra
- In caso di velocità nulla, scelta casuale

Questa policy è ben lontana da una policy ottima ma, non essendo l'obiettivo dello studio, si è deciso di optare per questa semplice policy il cui principio rimane quello dello sfruttare il movimento per guadagnare velocità e riuscire a raggiungere la bandiera in cima alla rampa. Al contrario di Cart Pole, Monte Carlo e Temporal Difference si differenziano in Mountain Car: Monte Carlo ha valutato solamente un gruppo ristretto di stati (quelli vicino all'inizio) mentre Temporal Difference invece ha effettivamente valutato stati esplorati dall'agente.

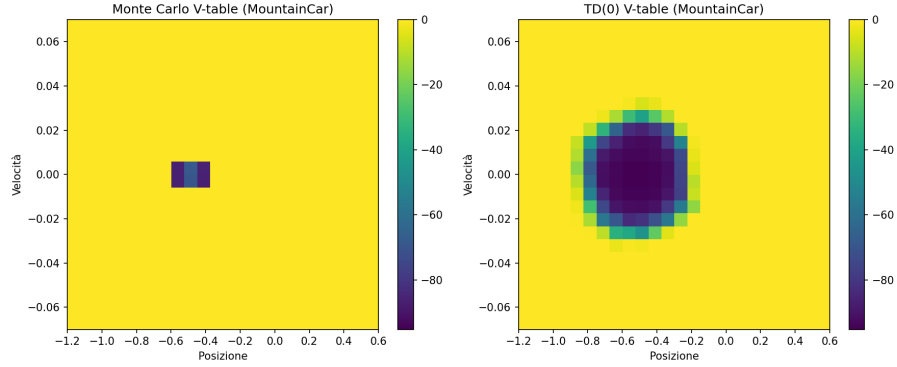


Figure 15: Heatmap MC vs TD

Considerando Temporal Difference, gli stati che si scontano di più dalla posizione iniziale sono valutati meglio e questo è in linea con come dovrebbe giocare un agente a Mountain Car: quando raggiunge una nuova posizione, significa che ho avuto abbastanza velocità per raggiungerla quindi la macchina sta effettivamente prendendo velocità, abbastanza per potersi avvicinare sempre di più alla bandierina. Se l'agente stesse eseguendo mosse puramente casuali, sarebbe molto più difficile ottenere questo movimento per accelerare e conseguentemente gli stati non sarebbero valutati correttamente. Usando invece questa policy, seppure non ottima, Temporal Difference ha assegnato dei valori in linea con la logica dell'ambiente. Una possibile spiegazione viene dalle scelte casuali che la policy usa: scegliendo casualmente quando la velocità è nulla, possiamo approssimare questa policy ad una epsilon greedy ma con "probabilità controllata", il che significa che questo comportamento di Monte Carlo potrebbe essere dovuto ai suoi problemi di varianza come sottolineato nella sezione 2.3.

4.2.3 Sensibilità α

Il parametro α sembra particolarmente influente per l'apprendimento di Mountain Car: sia SARSA che Q-Learning hanno difficoltà ad apprendere con un valore troppo alto (cfr. $\alpha = 0.5$, linea rossa dei grafici sotto) implicando che probabilmente i due algoritmi (specialmente Q-Learning) stiano rischiando di andare in "overshooting", ossia il passo che fanno è troppo grande per l'ottimo e lo superano. Questo è suggerito anche dai valori 0.1 e 0.2: entrambi ottengono risultati migliori ma si dimostrano oscillanti. Questa variazione tra un episodio e l'altro era visibile anche nel primo confronto (Figura 14) che usava un valore di α pari a 0.2. Il valore ideale sarebbe quindi compreso tra 0.05 e 0.2 in modo tale da avere la stabilità di 0.05 ma la rapidità d'apprendimento di 0.2, pur notando che 0.1 potrebbe essere troppo alto viste le oscillazioni.

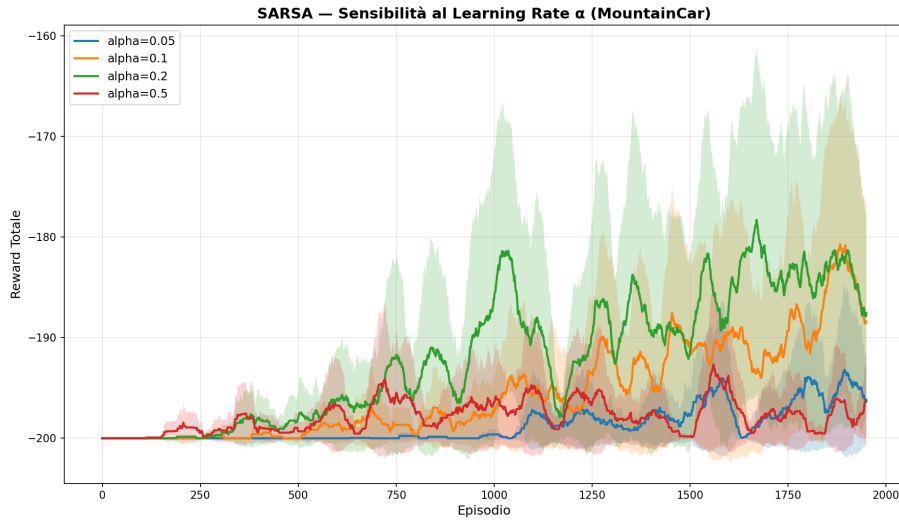


Figure 16: Sensibilità SARSA α

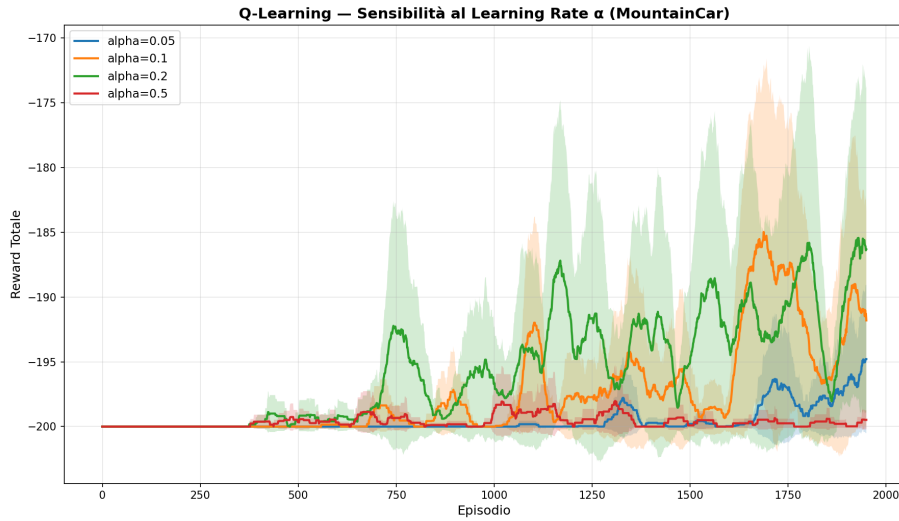


Figure 17: Sensibilità Q-Learning α

4.2.4 Sensibilità γ

I due algoritmi reagiscono diversamente alle variazioni di discount factor: logicamente, dovendo "dondolare" nell'insenatura per guadagnare velocità, tutte le azioni sono importanti perché la policy ottima conterà nell'oscillare e accelerare

nella direzione opposta per guadagnare la massima spinta per risalire la collina. Tuttavia, SARSA dimostra "indifferenza" verso γ : specialmente tra gli episodi 1300 e 1700 dove i valori (eccetto 0.9) si sovrappongono in termini di reward totale e anche negli episodi successivi questo è vero per γ pari a 0.99 e 1. Questo è vero anche per Q-Learning che però preferisce solo i due valori più grandi del discount factor e le sovrapposizioni di reward avvengono solo inizialmente.

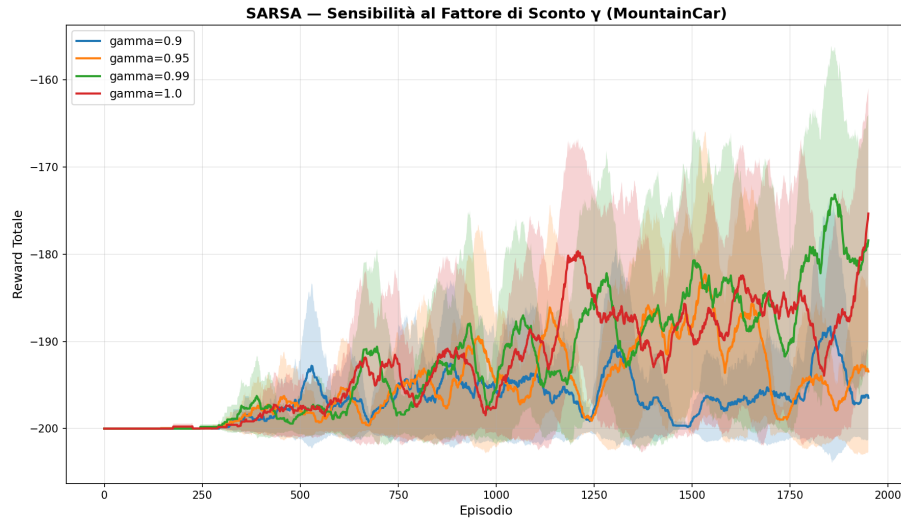


Figure 18: Sensibilità SARSA γ

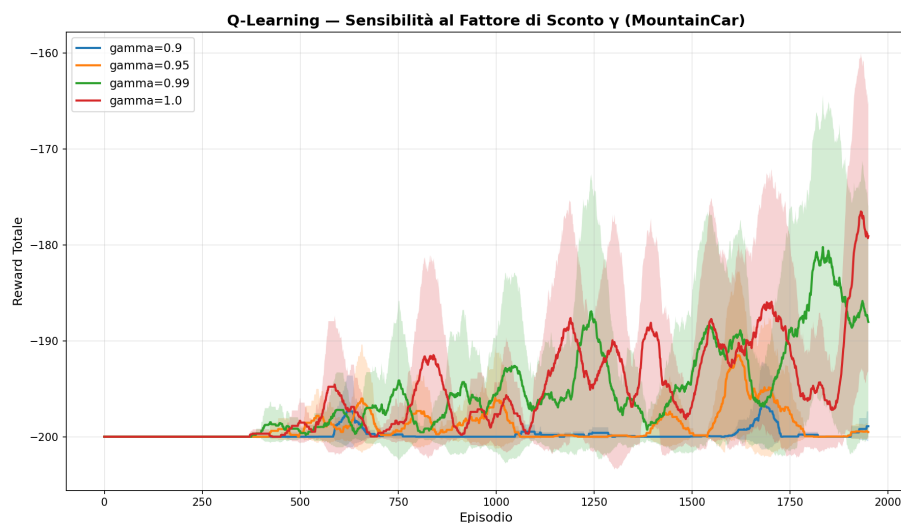


Figure 19: Sensibilità Q-Learning γ

4.2.5 Sensibilità ϵ

Al contrario di Cart Pole, ϵ sembra essere meno favorita: in un gioco dove la mia sequenza di azioni conta di più al fine di raggiungere il miglior risultato e con 3 azioni anziché 2, i valori di ϵ più piccoli sono preferiti. Il riscontro più lampante lo si ha mettendo a confronto i valori 0.01 e 0.3 per entrambi gli algoritmi: scegliendo un'azione casuale una volta su 3, l'agente fa molta fatica a raggiungere l'obiettivo. Invece, scegliendo casualmente 1 volta su 100 l'agente ha emno difficoltà a troncare l'episodio raggiungendo la bandiera mostrando che la policy sta venendo correttamente appresa e, conseguentemente, usata.

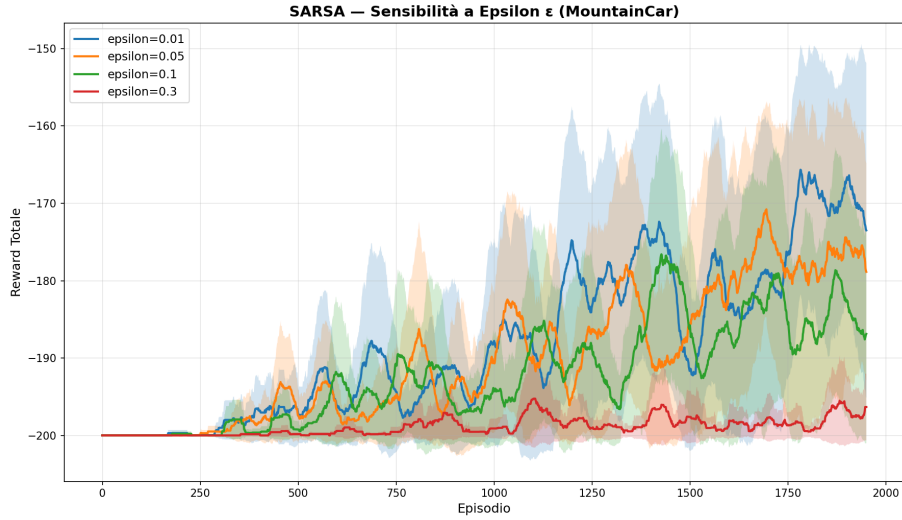


Figure 20: Sensibilità SARSA ϵ

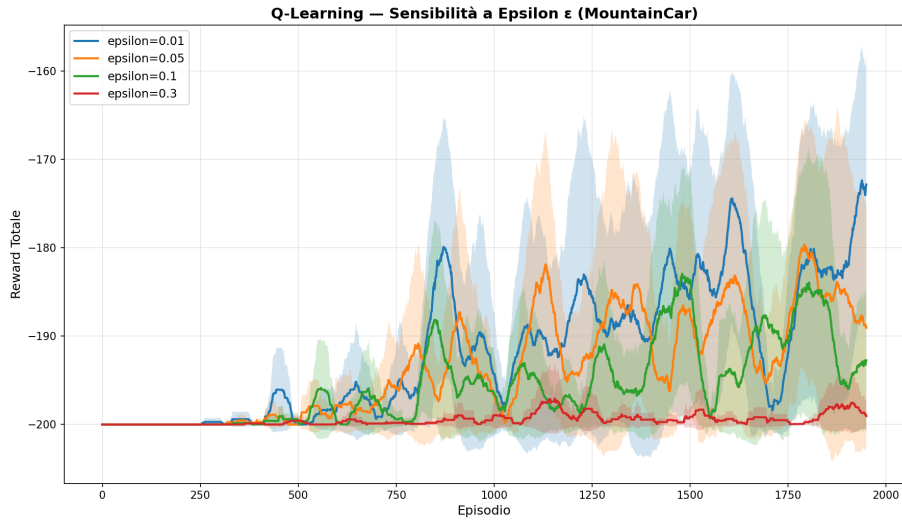


Figure 21: Sensibilità Q-Learning ϵ

4.3 Freeway

Gli esperimenti su Freeway hanno dapprima voluto esaminare la differenza tra le implementazioni di Deep Q-Network con e senza le tecniche descritte in 2.5. Di seguito i risultati con $\epsilon = 0.1$:

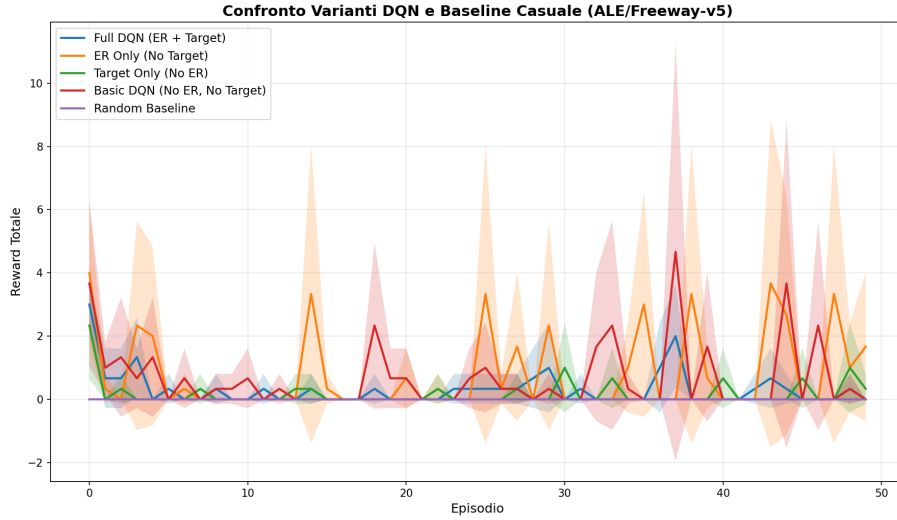


Figure 22: Varietà di risultati con ER e TN

Sebbene tutte le implementazioni risultino migliori della policy casuale, le diverse combinazioni di tecniche si dimostrano variegata: sorprendentemente, l'aggiunta sia di Experience Replay che Target Network non è l'implementazione migliore ma anzi si trova al di sotto della DQN "vanilla" e di quella con solo Experience Replay. In particolare, quest'ultima si dimostra la migliore arrivando anche ad oltre punti nel tempo limite. La Target Network da sola si dimostra appena migliore della policy casuale ma con comunque basse performance.

Di seguito la sensibilità della DQN completa ai valori di ϵ :

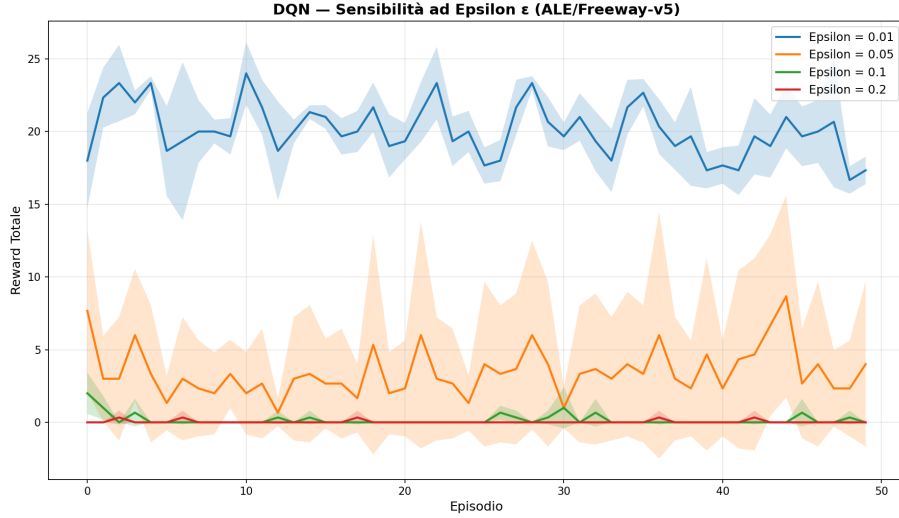


Figure 23: Sensibilità ad ϵ

I risultati evidenziano come la riduzione della probabilità della scelta casuale porta ad un drastico aumento di performance: con un ϵ di 0.01, l'agente riesce addirittura a superare i 25 punti. Questo suggerisce che l'agente sta imparando correttamente la policy da seguire ed è la casualità della scelta dell'azione ad ostacolarlo in una misura maggiore di quanto inizialmente aspettato.

5 Failures

Tra i limiti dello studio c'è anzitutto la limitata verifica di Monte Carlo e Temporal Difference: i due algoritmi usavano una solo policy per ogni ambiente. Per verificare effettivamente come i due algoritmi possano reagire con policy diverse, sarebbe stato necessario creare un numero sufficiente di policy per "fare statistica" (e.g.: 30) in modo da verificare, al di là dell'aspetto matematico, come i due algoritmi pesassero gli stati in rispetto del goal.

Il più grande limite viene da Freeway stesso: per poter eseguire gli esperimenti, è stato temporaneamente migrato il rispettivo codice su Colab per fare uso di GPU. I test hanno comunque richiesto un tempo complessivo di 4 ore, rendendo impraticabile eseguire lo stesso test su CPU su una macchina locale. Visti i limiti di tempo del servizio, non è stato possibile eseguire test più approfonditi, ad esempio una combinazione della variazione di ϵ con le diverse implementazioni.

6 Conclusioni

6.1 Aspettative

Cart Pole e Mountain Car hanno avuto risultati molto simili a quelli che ci si aspettava: l'apprendimento iniziale si è rivelato lento ma, con il giusto numero di episodi, l'agente ha cominciato a "saper giocare" e concludere l'obiettivo. Monte Carlo e Temporal Difference hanno valutato gli spazi negli stati valutando meglio quegli stati che avvicinano al goal.

6.2 Le sorprese

Freeway è l'ambiente che ha destato più sorpresa: inizialmente si pensava che l'aggiunta sia di Experience Replay che di Target Network sarebbe stata l'implementazione con i maggiori risultati ma non è stato così. Un'altra inaspettata scoperta viene dalla variazione di ϵ , sempre in Freeway: si pensava che la differenza sarebbe stata poco significativa ma si è invece rivelata essere di grande impatto, portando ad un aumento vertiginoso dei risultati.

References

- [1] Gymnasium Website (<https://gymnasium.farama.org/>)
- [2] Arcade Learning Environment (<https://ale.farama.org/>)
- [3] Cart Pole Environment, Gymnasium Website (https://gymnasium.farama.org/environments/classic_control/cart_pole/).
- [4] Mountain Car Environment, Gymnasium Website (https://gymnasium.farama.org/environments/classic_control/mountain_car/).
- [5] Freeway Environment, ALE website <https://ale.farama.org/environments/freeway/>.